



UNIVERSITATEA DIN BUCUREȘTI

**FACULTATEA DE MATEMATICĂ ȘI
INFORMATICĂ**

SPECIALIZAREA INFORMATICĂ



Lucrare pentru cursul de procesare al semnalelor
COMPRESIA LZ77 ȘI LZ78

Student

Alexandru Iacob

Grupa 333

Profesor coordonator

Cristian Rusu

București, februarie 2026

CUPRINS

1. Introducerea	3
2. Noțiuni de bază	4
3. LZ77	5
4. LZ78	8
5. Comparatie de eficiență.....	9
6. Concluzii	11
Bibliografie	12

FIGURI

Figură 1 Comparatie a ratei de compresie pentru $min_match = 2$ și $min_match = 5$	7
Figură 2 Comparatie timpului de codare pentru fiecare $min_match = 2..5$	7
Figură 3 Rata de compresie în funcție de <i>windowLength</i>	8
Figură 4 Pareto scatter pentru detectarea soluției optime	8
Figură 5 Comparatie între algoritmii LZ77 și LZ78	10

1. INTRODUCEREA

În studiul informaticii, compresia datelor face referire la reducerea dimensiunii fișierelor, folosind mai puțini biți de memorie (MAHDI, MOHAMMED, & MOHAMMED, 2012). Aceasta se poate realiza fie cu pierdere de informații (*lossy*), sau fără a pierde date (*lossless*). Compresia *lossless* are ca bază de pornire identificarea și eliminarea redundanțelor relative ale fișierului prelucrat. Compresia de tip *lossy*, în schimb, reduce numărul de biți prin indentificarea și eliminarea porțiunilor inutile sau care sunt mai puțin relevante (JAGADISH H. PUJAR, 2010). De cele mai multe ori, dispozitivul care efectuează compresia datelor poartă numele de *encoder*, în vreme ce cel care execută operația inversă (decompresia) se numește *decoder*.

Principalele avantaje ale comprimării datelor constă în reducerea folosirii memoriei externe și a timpului de transmisie. Fișierele comprimate ocupă mult mai puțin spațiu în memorie, ceea ce duce la o reducere semnificativă a costurilor privitoare la stocare. Mai mult, un fișier comprimat ocupă mai puțină lungime de bandă în rețea fapt care duce la un timp scăzut de transfer (KIMURA & LATIFI, 2005).

În schimb, un dezavantaj al comprimării datelor importante este reprezentat de folosirea mai multor resurse ale calculatorului. Din acest motiv se caută metode de optimizări ai algoritmilor de compresie prioritizând viteza și eficiența acestora.

Unii dintre algoritmii care stau la baza a numeroși algoritmi de compresie, cum ar fi cel folosit pentru fișierele GIF, PNG și ZIP, sunt LZ77 și LZ78. Aceștia sunt doi algoritmi de compresie de tip *lossless* publicați în lucrările lui Abraham Lempel și a lui Jacob Ziv din anii 1977 (ZIV & LEMPEL, A Universal Algorithm for Sequential Data Compression, 1977) și 1978 (ZIV & LEMPEL, Compression of individual sequences via variable-rate coding, 1978).

În teorie, LZ77 și LZ78 sunt „codificatoare bazate pe dicționar” (*dictionary coders*). LZ77 gestionează o fereastră glisantă pe timpul compresiei, în timp ce LZ78 se bazează pe crearea unui dicționar explicit. Cei doi algoritmi s-au dovedit a fi echivalenți doar când se vrea decompresia întregului set de date.

În timp ce codificarea LZ77 se folosește de caracterele vizitate anterior, decodificarea trebuie să înceapă neapărat de la începutul inputului. În principiu, decompresia LZ78 ar putea permite accesarea aleatorie dacă întreg dicționarul ar fi știut de la început, dar în practică acest

lucru este destul de improbabil, dicționarul creându-se în timpul codării/decodării (CHANG, FERNANDO, & HU, fără an).

2. NOȚIUNI DE BAZĂ

În utilizarea algoritmilor LZ77 și LZ78 se folosesc mai multe concepte cum ar fi redundanța, entropia, dicționare, fereastră glisantă și altele. Mai departe voi trece în revistă aceste concepte și ce descriu ele pentru a putea fi mai ușor de urmărit ulterior.

Datele care sunt prelucrate și cu care avem de a face conțin de cele mai multe ori informații redundante și repetitive. Aici, redundanța se referă la diferența dintre entropia $H(X)$ a unui set de date X și valoarea maximă posibilă $\log(|\mathcal{A}_X|)$. Practic, este „spațiul” utilizat inutil pentru a transmite date. Din acest motiv se folosesc diverși algoritmi de compresie, cu toate că uneori redundanța este folosită intenționat pentru corectarea greșelilor viitoare dacă datele sunt transmise pe un canal cu zgomot sau sunt corupte în vreun fel.

Entropia unui set de date descrie nivelul de informație asociat cu acesta. Ea măsoară cantitatea de informație necesară pentru a descrie starea unei variabile, considerând distribuția de probabilități pe toate stările posibile. Fiind dată o variabilă aleatorie discretă X care poate fi orice membru x al mulțimii $\mathcal{X}$ și care are distribuția $p: \mathcal{X} \rightarrow [0,1]$, atunci entropia va fi

$$H(X) := - \sum_{x \in \mathcal{X}} p(x) \log(p(x)).$$

Alegerea bazei pentru logaritm variază în funcție de aplicație, dar cea mai utilizată în informatică este baza 2 care returnează biți de informație. Acest concept de entropie a fost prima dată introdus de Claude Shannon în 1948 (SHANNON, 1948).

Dicționarul folosit de algoritmul LZ78 reprezintă o structură de date care reține o mulțime de stringuri. Mai precis, dicționarele sunt niste tipuri de date abstracte care stochează colecții de tip cheie-valoare, astfel încât fiecare cheie este unică. Din punct de vedere matematic este o funcție cu un domeniu finit (COLLINS & SYME, 1995).

Noțiunea de „ferastră glisantă” se referă la o modalitate de a rezolva diverse probleme în care intervin subșiruri. În loc să se parcurgă în mod repetat iterativ aceleași elemente, fereastra glisantă restricționează setul de date la un interval fix. Astfel, ideea principală prin care

funcționează fereastra glisantă este că se folosește de rezultatul ferestrei anterioare și a datelor curente fără să mai verifice toate datele.

Un *stream* este o secvență de date, potențial infinită, care devine valabilă cu timpul. Acesta poate fi interpretat ca o bandă rulantă ce aduce informație, aceasta urmând să fie procesată pe rând, în loc de toată deodată.

3. LZ77

Algoritmul LZ77 a fost prima dată publicat în anul 1977 de către Abraham Lempel și Jacob Ziv. Deși nu a fost primul algoritm de compresie existent, acesta a fost inovator prin folosirea unei ferestere glisante, fapt fără precedent până atunci. Introducerea acestuia a permis procesarea arbitrară a unor porțiuni de date într-un mod optim, fără a mai fi nevoie de verificarea întregului set de date până în punctul procesării.

Algoritmii de tip LZ77 reușesc să ajungă la compresia datelor prin înlocuirea aparițiilor repetate cu referințe la o copie deja existentă în datele originale. O potrivire va fi codificată prin perechi de numere *lungime (l)* – *distanță (d)*, care vor fi interpretate ca „fiecare caracter pe următoarea *lungime (l)* sunt exact aceleași ca cele de la aflate la *distanță (d)* înapoi în streamul necomprimat”.

Pentru a putea face potriviri, programul trebuie să rețină o parte din datele cele mai recente. În practică se rețin ultimi k biți, unde k de cele mai multe ori o putere a lui 2 pentru rapiditate. Acest lucru e gestionat de fereastra glisantă. Evident, cu cât fereastra aleasă este mai mare/reține mai multe informații, algoritmul va căuta potriviri cu date mult mai depărtate de punctul curent.

În prezenta lucrare voi folosi următorul algoritm pentru codarea LZ77:

```
cât timp mai pot să preiau din input execută
|   match := cel mai lung sir din look_ahead_buffer care se repetă în
search_buffer
|   dacă match există atunci
|   |       d := distanța cu care trebuie să mă întorc pentru a copia
match
|   |       l := lungimea șirului match
|   |       c := caracterul care urmează după șirul copiat
|   | altfel
|   |       d := 0
|   |       l := 0
```

```

|      |      c := 0
|      └─
|
|      output (d,l,c)
|      elimină l+1 caractere de la începutul ferestrei (search_buffer)
|      s := pop l+1 caractere de la începutul inputului
|      (look_ahead_buffer)
|      adaugă s la finalul search_bufferului
|      └─

```

Eficiența algoritmului este dată, deci, de alegerea potrivită a parametrilor pentru a avea o compresie cât mai bună într-un timp cât mai scurt. Astfel, am ales ca parametri de optimizat: *windowLength*, *searchBufferProcent* și *minMatch*. Aici, *minMatch*-ul reprezintă limita inferioară de căutare a unui subșir, un fapt care eficientizează compresia prin spațiul de memorie stocat și timpul de căutare.

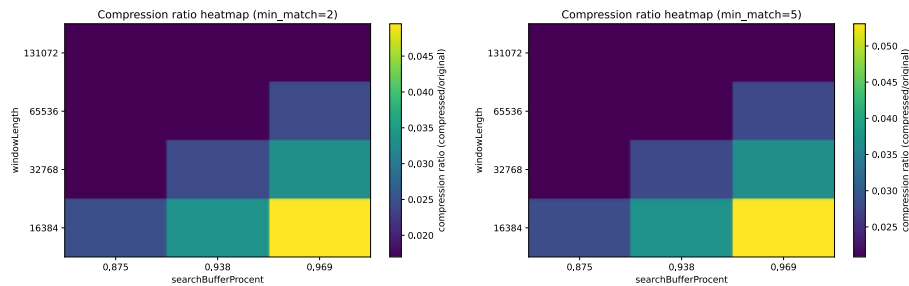
Pe eșantionul de date [2,3,4,5] și un fișier de 338,366 bytes (3KB), *minMatch* pare să dea un rezultat similar în privința ratei de compresie¹ (Figura 1). În schimb timpul de compresie diferă de la o valoare la alta (Figura 2). Evident, pentru o comprimare mai bună este nevoie de un timp mai lung de rulare. Se observă din grafice că valoarea optimă pentru *min_match* ar fi între 3 și 4 dat fiind faptul că se apropie cel mai mult de un echilibru între viteză și precizie.

Mai departe am analizat cum este influențată rata de compresie în funcție de *windowLength* și de procentul pe care îl alocăm pentru *search_buffer*. Similar ca anterior pentru valori diferite ale *min_match*ului graficele sunt similare, fapt pentru care voi avea în evidență în continuare *min_match* = 3 (Figura 3). Din grafic se observă că pentru un procent 88% (7/8) din lungimea ferestrei alocat pentru *search_buffer* este mai eficient decât celelalte variante per total. Cu toate acestea pentru dimensiuni ale ferestrei mai mari decât 2^{15} bytes nu mai face o așa de mare diferență. Mai mult, o dimensiune mai mare decât 2^{14} bytes a ferestrei nu mai e relevantă dat fiind faptul că după aceea raportul scade foarte puțin.

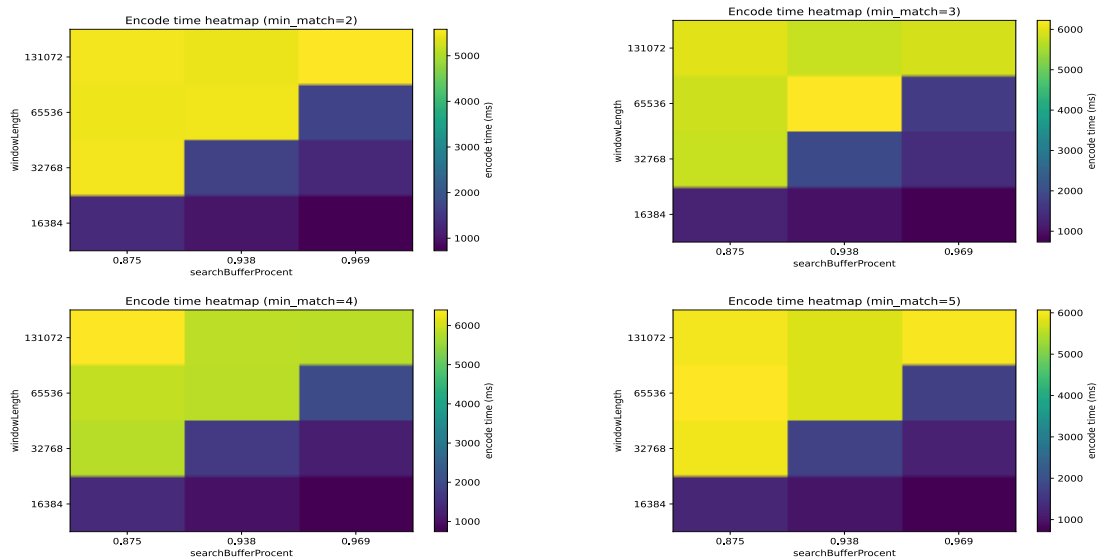
În continuare am decis să fac un Pareto scatter în funcție de timpul de compresie. Acest tip de graf este folosit pentru detectarea zonelor de interes și pentru controlul calității. Din grafic se observă că cea mai bună compresie este pentru tuplul (*min_match* = 2, *windowLength* = 2^{15} , *searchBufferProcent* = 0.875). Acest lucru se susține și din figurile 1 și 2, unde pentru aceeași

¹ Pe parcursul acestei lucrări rata de compresie se calculează ca $\text{numar_de_biti_din_compresie} / \text{numar_de_biti_original}$

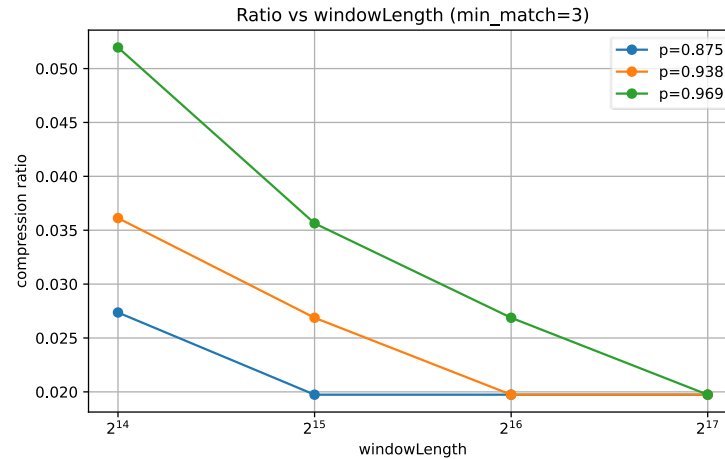
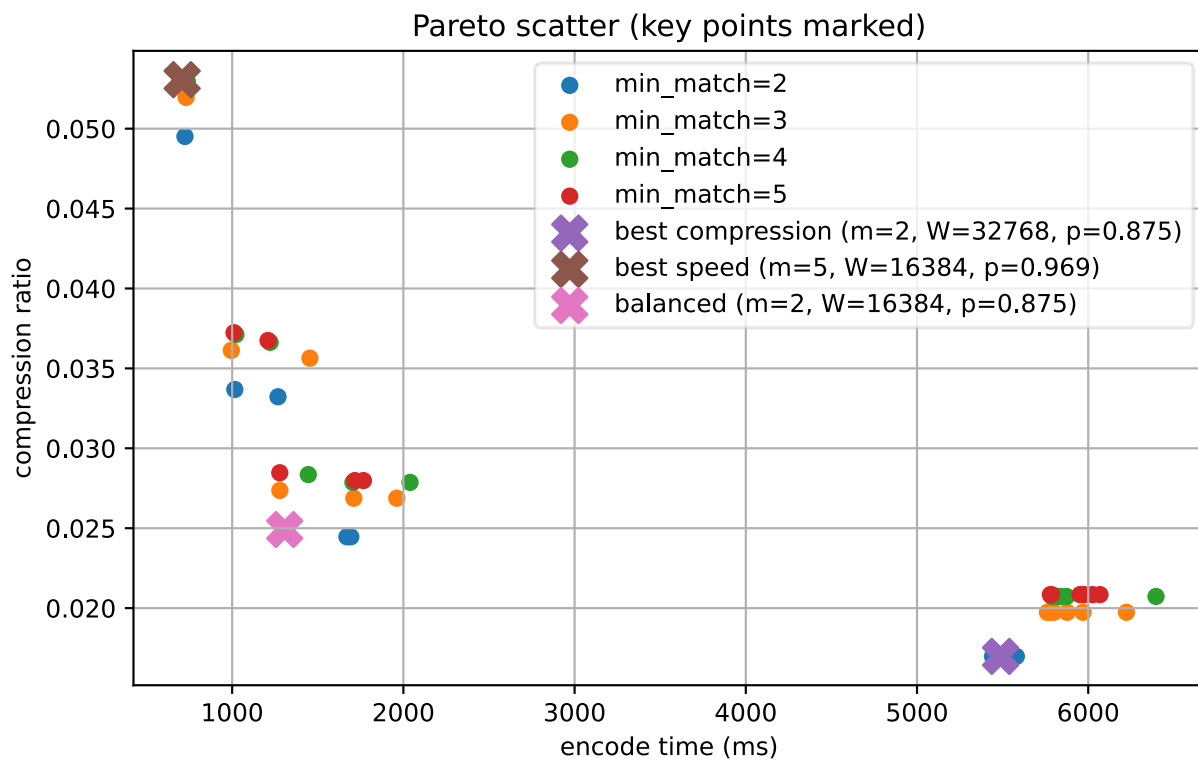
parametrii obținem rezultatele pentru cea mai bună comprimare. Cu toate acestea, detrimentul acestei configurații este determinat de timpul lung de rulare. Dacă vrem ca algoritmul să funcționeze la viteză mare atunci vom folosi ca parametrii ($min_match = 5$, $windowLength = 2^{14}$, $searchBufferProcent = 0.969$). Pentru un rezultat care să fie optim – rapid și cu rata de compresie cât mai mică – parametrii potriviți sunt ($min_match = 2$, $windowLength = 2^{14}$, $searchBufferProcent = 0.875$) care reprezintă o combinație a celor două rezultate anterioare.



Figură 1 Comparație a ratei de compresie pentru $min_match = 2$ și $min_match = 5$



Figură 2 Comparație timpului de codare pentru fiecare $min_match = 2..5$

Figură 3 Rata de compresie în funcție de *windowLength*

Figură 4 Pareto scatter pentru detectarea soluției optime

4. LZ78

Algoritmii LZ78 de compresie (ZIV & LEMPEL, Compression of individual sequences via variable-rate coding, 1978) se bazează pe crearea/existența unui dicționar de tokenuri. Acești algoritmi reprezintă un *n-ar tree*, unde *n* este numărul de tokenuri folosit pentru a forma secvențe.

Fiecare intrare din dicționar este de tipul *dicționar[...]* = {*indice*, *token*}, unde indicele reprezintă o secvență deja găsită, iar tokenul este următorul caracter. Se observă că algoritmul nu adaugă intrări noi, decât dacă acea intrare este unică.

Pseudocodul algoritmului pe care l-am utilizat este:

```

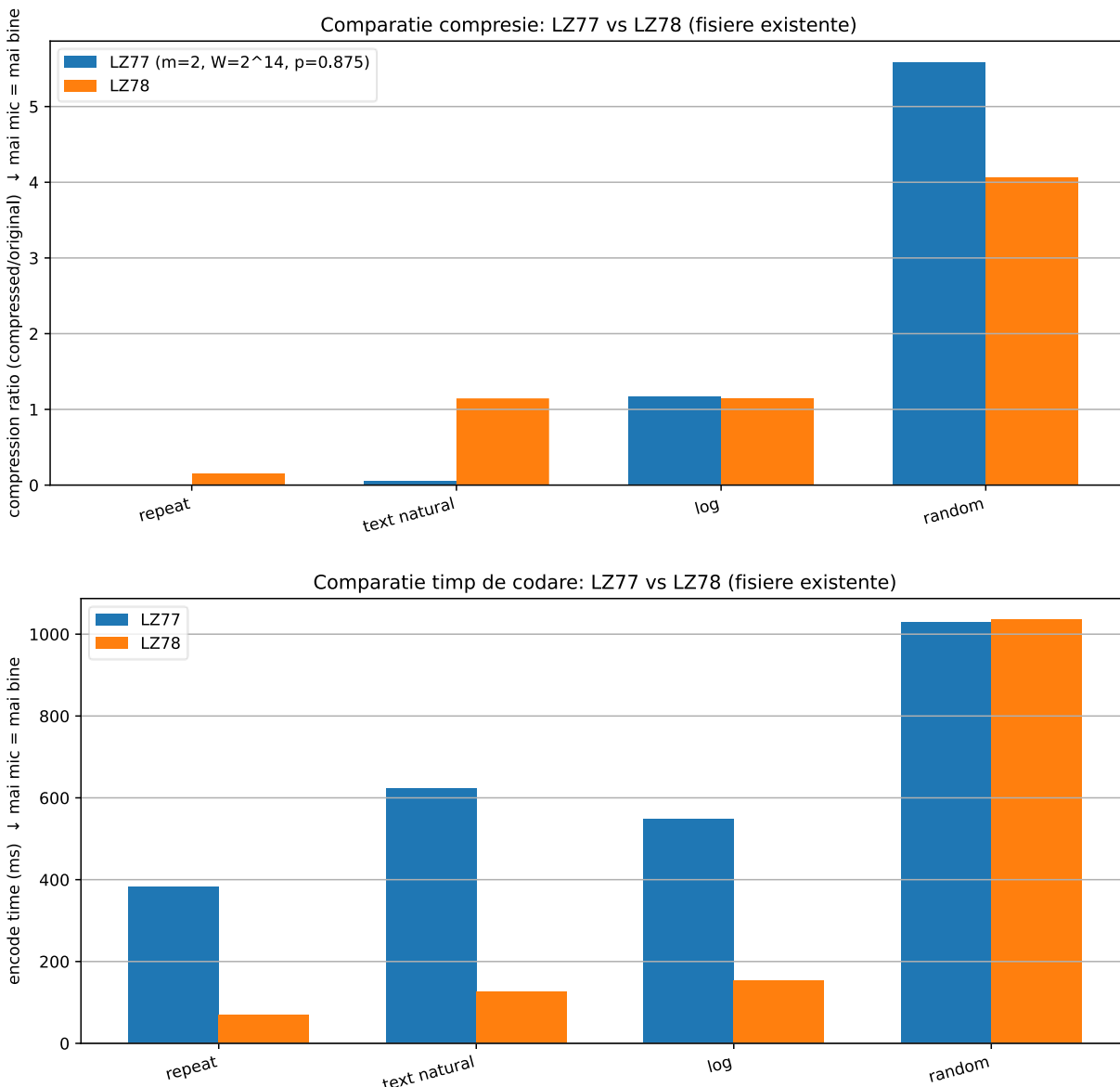
input ← citește fișierul ca string
input ← input + ','
tree ← dicționar gol
index ← 1
output ← listă cu (None, ',')
i ← 0
cât timp i < len(input) execută
|   parent ← 0
|   cât timp tree conține cheia parent execută
|   |   găsit ← FALS
|   |   pentru fiecare (child, token) din tree[parent] execută
|   |   |   dacă token = input[i] atunci
|   |   |   |   parent ← child
|   |   |   |   i ← i+
|   |   |   |   găsit ← ADEVĂRAT
|   |   |   |   break
|   |   |   └─┐
|   |   └─┐
|   └─┐
|   dacă găsit = FALS atunci
|   |   adaugă (index, input[i])
|   |   break
|   └─┐
|   dacă tree NU conține cheia parent atunci
|   |   tree[parent] ← [(index, input[i])]
|   └─┐
|   adaugă (parent, input[i]) la output
|   index ← index+1
|   i ← i+1
└─┐
return output

```

5. COMPARAȚIE DE EFICIENȚĂ

Având ambii algoritmi funcționali am putut să fac un test să compar eficiența celor doi algoritmi și să văd care ar fi „mai bun”. Am testat fiecare algoritm de câte 3 ori pe fiecare dintre cele patru fișiere de aproximativ 264KB de tipuri diferite – unul cu repetiție constantă, unul cu

limbaj natural, unul cu logs și unul cu text complet random (pentru *worst-case scenario*)² (Figura 5). Astfel am observat că în ce privește compresia LZ77 este mai eficient la textele repetitive și textele naturale majoritar prin acea fereastră glisantă care găsește mai bine potrivirile, LZ78 întrecându-l doar pe testele de fișiere log și random. În ce privește timpul de rulare, LZ78 este mult mai rapid decât LZ77.



Figură 3 Comparăție între algoritmi LZ77 și LZ78

² Fișierele au fost generate cu ChatGPT pentru testare mai rapidă.

6. CONCLUZII

În această lucrare am analizat compresia *lossless* prin doi algoritmi fundamentali, LZ77 și LZ78, evidențiind atât mecanismele lor (fereastră glisantă vs. dicționar explicit), cât și impactul alegerii parametrilor asupra performanței. Pentru LZ77, experimentele au arătat clar că eficiența depinde puternic de configurarea *windowLength*, *searchBufferProcent* și *min_match*: valori mai permissive tind să îmbunătățească rata de compresie, dar cresc timpul de codare. Folosind graficele comparative și analiza de tip Pareto scatter, am identificat configurații care favorizează fie compresia maximă, fie viteza, fie un compromis rezonabil între cele două.

Comparația directă dintre algoritmi pe patru tipuri de fișiere text (repetitiv, limbaj natural, log-uri structurate și text pseudo-random) indică faptul că nu există un “câștigător universal”: LZ77 tinde să fie mai eficient la compresie pe texte repetitive și naturale, datorită potrivirilor locale găsite eficient în fereastra glisantă, în timp ce LZ78 poate depăși LZ77 pe date unde structura favorizează dicționarul (ex. log-uri) și rămâne competitiv chiar și în scenarii nefavorabile (random), având totodată un timp de rulare semnificativ mai mic în implementarea testată. În concluzie, alegerea între LZ77 și LZ78 trebuie făcută în funcție de tipul datelor și de prioritatea aplicației (compresie vs. viteză). Ca direcții de îmbunătățire, o evaluare și mai realistă ar include o serializare binară a codului (pentru a evita overhead-ul reprezentării textuale a tuplurilor) și optimizări ale căutării (structuri de date mai eficiente pentru match-uri / dicționar), pentru a compara algoritmi mai aproape de condițiile unui compresor practic.

BIBLIOGRAFIE

- CHANG, E., FERNANDO, U., & HU, J. (n.d.). *Lossless Data Compression*. Retrieved from Data Compression: A General Study: <https://cs.stanford.edu/people/eroberts/courses/soco/projects/data-compression/lossless/lz78/concept.htm>
- COLLINS, G., & SYME, D. (1995). A theory of finite maps. *Higher Order Logic Theorem Proving and Its Applications*.
- JAGADISH H. PUJAR, L. M. (2010). A NEW LOSSLESS METHOD OF IMAGE COMPRESSION AND DECOMPRESSION USING HUFFMAN CODING TECHNIQUES. *Journal of Theoretical and Applied Information Technology*.
- KIMURA, N., & LATIFI, S. (2005, April 4). A Survey on Data Compression in Wireless Sensor Networks. *Proceedings of the International Conference on Information Technology: Coding and Computing (ITCC'05)*.
- MAHDI, O. A., MOHAMMED, M. A., & MOHAMMED, A. J. (2012). Implementing a Novel Approach an Convert Audio Compression to Text Coding via Hybrid Technique. *IJCSI International Journal of Computer Science Issues*.
- SHANNON, C. (1948). A Mathematical Theory of Communication. *Bell System Technical Journal*.
- ZIV, J., & LEMPEL, A. (1977). A Universal Algorithm for Sequential Data Compression. *IEEE Transactions on Information Theory*.
- ZIV, J., & LEMPEL, A. (1978). Compression of individual sequences via variable-rate coding. *IEEE Transactions on Information Theory*.